DriveDex v2.0

Update specification & build guide for Claude Code

Faster copying from phones and drives: plan from the index, stream from the phone, measure everything.

Field	Value
Project	DriveDex — github.com/YashwardhanV/DD (Go, Windows, no CGO)
Owner	Yashwardhan Verma
Base version	v1.0.0 + post-release work up to commit `31d50e9` (USB speed check)
Target version	v2.0.0
Document date	3 November 2026
Audience	Claude Code (builds it) and the owner (runs hardware checks)
Owner's goals	1) Copying is slow in real use. 2) Phone → PC must be much faster. 3) Any other safe speed-up.

How to hand this off. Put this PDF in the repo as `docs/BUILD-V2.pdf` and tell Claude Code: *"Read docs/BUILD-V2.pdf completely, then build DriveDex v2.0 stage by stage, starting at Stage 13. Follow section 0 of BUILD.md as well."* Everything Claude Code needs — what to build, why, the architecture, the Go contracts, the tests and the acceptance checks — is in this one file.

Contents

0. Instructions for Claude Code (read first) . . . 3

1. Why v2.0: the problem and what v1 already does . . . 3

1.1 The owner's report . . . 3

1.2 What v1 already does well (keep all of it) . . . 4

1.3 What is actually slow — gaps found in the v1 code . . . 4

2. Where copy time goes (the model behind every stage) . . . 5

3. v2.0 at a glance . . . 5

4. Architecture of the v2.0 copy engine . . . 6

4.1 Components . . . 6

4.2 How a copy runs in v2.0 . . . 7

4.3 Choosing the plan source . . . 7

5. Shared contracts (Go types, config, flags) . . . 8

5.1 Tracer — internal/copier/trace . . . 8

5.2 Plan sources and the reconciler — internal/copier . . . 9

5.3 The shared atomic writer . . . 9

5.4 Phone — internal/phone . . . 9

5.5 Autotuner — internal/copier/tune.go . . . 10

5.6 New configuration keys (config.toml) . . . 10

5.7 New and changed CLI surface . . . 10

6. Stages . . . 11

Stage 13 — Copy tracer and bench copy . . . 11

Stage 14 — Phone USB link check . . . 12

Stage 15 — Phone: plan from the index, list only what changed . . . 13

Stage 16 — Phone: tar stream lane (the main phone → PC speed-up) . . . 13

Stage 17 — Local copy: plan from the index + reconciling sweep . . . 15

Stage 18 — Local fast paths . . . 16

Stage 19 — Autotuner . . . 17

Stage 20 — Disk-order reads for hard-disk sources . . . 17

Stage 21 — Docs, benchmarks and the v2.0.0 release . . . 18

7. Benchmark protocol (how “faster” is decided) . . . 18

8. Risks, fallbacks and what is out of scope . . . 19

8.1 Risks . . . 19

8.2 Out of scope for v2.0 (candidates for v2.1) . . . 19

Appendix A — PROGRESS.md section to add . . . 20

Appendix B — Files touched (map for Claude Code) . . . 20

Appendix C — Glossary for the owner . . . 21

0. Instructions for Claude Code (read first)

This document extends `BUILD.md`. **Every rule in BUILD.md section 0 still applies** (no CGO, ask before installs/admin/registry/push, read-only on indexed drives, heavily commented code, `context.Context` everywhere, PROGRESS.md kept up to date). The additions below are specific to v2.0.

1. **Read the whole PDF once before writing code.** Then read `PROGRESS.md`, `internal/copier/*.go`, `internal/phone/*.go` and `internal/cli/copy.go` — v2.0 changes these the most.
2. **Stages 13 → 21, strictly in order.** Stage 13 (measurement) comes first on purpose: every later speed feature is judged with it.
3. **Every speed feature ships behind a switch and has a fallback to the v1 path.** If its benchmark does not show a gain on the owner's hardware, it stays *off by default* (the flag remains) and PROGRESS.md says why. Never claim a speed-up that was not measured on real hardware.
4. **Never break v1 behaviour.** Resume, atomic writes (`.ddxpart`), conflict policies, FAT32 4 GiB check, free-space check, safe Windows names for phone files, the copy-into-itself refusal: all keep working and keep their tests.
5. **At the end of every stage**: `go build ./...`, `go vet ./...` (linux, windows/amd64, windows/arm64), `go test ./...`, and `go test -race ./internal/copier/... ./internal/phone/...`. All clean.
6. **Commit per stage**: `stage 13: copy tracer and bench copy`, etc. Work on a branch v2; ask the owner before pushing or opening a PR.
7. **Hardware checks** (phone, pen drive, USB hard disk) are listed under each stage as *Owner to verify*. Never mark them passed without the owner's pasted output. Put the list in PROGRESS.md as v1 did.
8. **Beginner-friendly reporting.** After each stage tell the owner what was built, the exact command to try, and the one concept it teaches (2–3 sentences).
9. **If this spec is wrong** (an API behaves differently, a phone lacks a command), don't hack around it silently: record it under *Deviations* in PROGRESS.md, pick the closest working approach, tell the owner.

> **Two new safety rules for v2.0.** (1) `bench copy` may delete only the folder it created itself (`<dst>\.drivedex-bench-<timestamp>`) and must check the path prefix before deleting anything. (2) DriveDex stays **read-only on the phone**: only `find`, `stat`, `tar -c` (create, to stdout) and `adb pull`. Never push files, never write a temp file on the phone.

1. Why v2.0: the problem and what v1 already does

1.1 The owner's report

- **Copying is slow in real use** — on the owner's own drives, not in the benchmark container.
- **The case that matters most is phone → PC** (camera roll, WhatsApp media, downloads).
- Goal: make DriveDex measurably faster than v1, and prove it with numbers from real hardware.

1.2 What v1 already does well (keep all of it)

Area	v1 behaviour	Where
Index	Saved .ddx per drive/phone, searched in ms; NTFS turbo mode (MFT + USN journal)	internal/index, internal/ntfs
Phone listing	One `find … -exec stat` on the phone instead of MTP	internal/phone/scanner.go
Phone copy	`adb pull -a` in batches of ≤100 files from one folder, 2 batches at once, via a hidden staging folder	internal/copier/phone.go
Local copy	Parallel workers (auto 2 / 4 / 8 from HDD-vs-flash), largest file first, 2 big-file lanes on USB, 1 MB buffers, preallocation ≥1 MB	internal/copier/copy.go, cli/copy.go
Safety	.ddxpart + rename, resume skips finished files, FAT32 and free-space pre-checks, CRC-32C --verify	internal/copier
Diagnosis	doctor, doctor --bench, USB 2.0-port detection (ports, drives)	internal/doctor, internal/volume

1.3 What is actually slow — gaps found in the v1 code

These are facts read from the v1 source (commit 31d50e9), not guesses:

#	Gap	Effect	v2 fix (stage)
G1	`drivedex copy` lists the source again (walk, or `find` on the phone) even when a fresh index of it exists	Nothing is copied until a full listing of a slow USB drive / phone finishes	Plan from the index (15, 17)
G2	Phone files move through `adb pull`: even in a batch, adb's sync protocol requests and acknowledges *each file*	Many small files (WhatsApp, screenshots) are slow; per-file round trips dominate	tar stream lane (16)
G3	Worker count is chosen once (2, 4 or 8) and never adjusted	Wrong for many real drives (cheap flash, SMR disks, USB 2.0)	Autotuner (19)
G4	Each source file is looked up twice: `os.Lstat` then `os.Open`	One extra metadata round trip per file on a USB drive	Single-open source (18)
G5	Big files go through the Windows cache	Fast at first, then stalls while the cache flushes; RAM pressure	Unbuffered big-file I/O (18)
G6	`--verify` may read the copy back from RAM (v1 says so in its help text)	Verify checks the copy logic, not the media	Unbuffered verify read (18)
G7	Small files on a hard disk are read in size/name order, not disk order	The head seeks constantly	Disk-order reads (20)
G8	Nothing tells the user where copy time goes, and no real-hardware numbers exist (BENCHMARKS.md section 2 is empty)	“Slow” can't be diagnosed or proven fixed	Tracer + bench copy (13)
G9	The USB-speed check covers drives, not phones; many phones only have USB 2.0	A 40 MB/s ceiling looks like a DriveDex problem	Phone link check (14)

The honest ceiling. No software beats the slowest link: the phone's storage, the USB port and cable (USB 2.0 ≈ 35–40 MB/s in practice), or the destination disk. v2.0 attacks the time spent *around* the bytes: listing, per-file round trips, waiting, seeking. For one 4 GB video the gain will be small; for 20,000 small files it can be large. Stage 13 proves which case each slow copy is.

2. Where copy time goes (the model behind every stage)

A copy of *N* files totalling *S* bytes costs roughly:

```
T  ≈  T_list  +  N × t_per_file  +  S / bandwidth  +  seeks (HDD only)

T_list      listing the source (walk / find)          -> removed by planning from the index
t_per_file  open, create, set time, close, rename,    -> fewer round trips, streaming,
            antivirus scan, adb request/ack               right number of workers
S/bandwidth bytes over the slowest link               -> can only be detected and explained
seeks       head movement on spinning disks           -> disk-order reads
```

File mix	What dominates	What helps most
Few big files (videos, ISOs)	S / bandwidth	Correct link (USB 3 port), unbuffered I/O, 1–2 streams
Many small files (photos, WhatsApp, docs)	N × t_per_file	tar stream (phone), more workers on flash, fewer round trips
Anything on a USB hard disk	seeks + per file	1–2 workers, disk-order reads
Huge folder, copy re-run (backup)	T_list + checking the destination	Plan from index, skip unchanged quickly

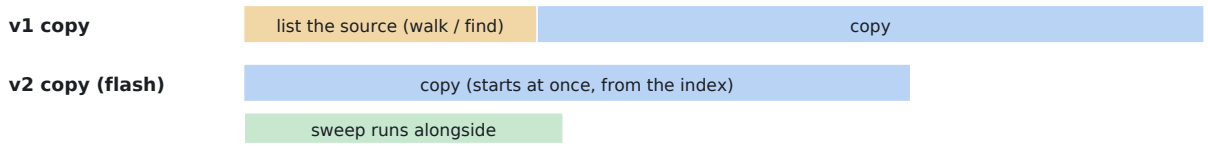


Illustration only — real proportions come from 'drivedex copy --trace' on your drives.

3. v2.0 at a glance

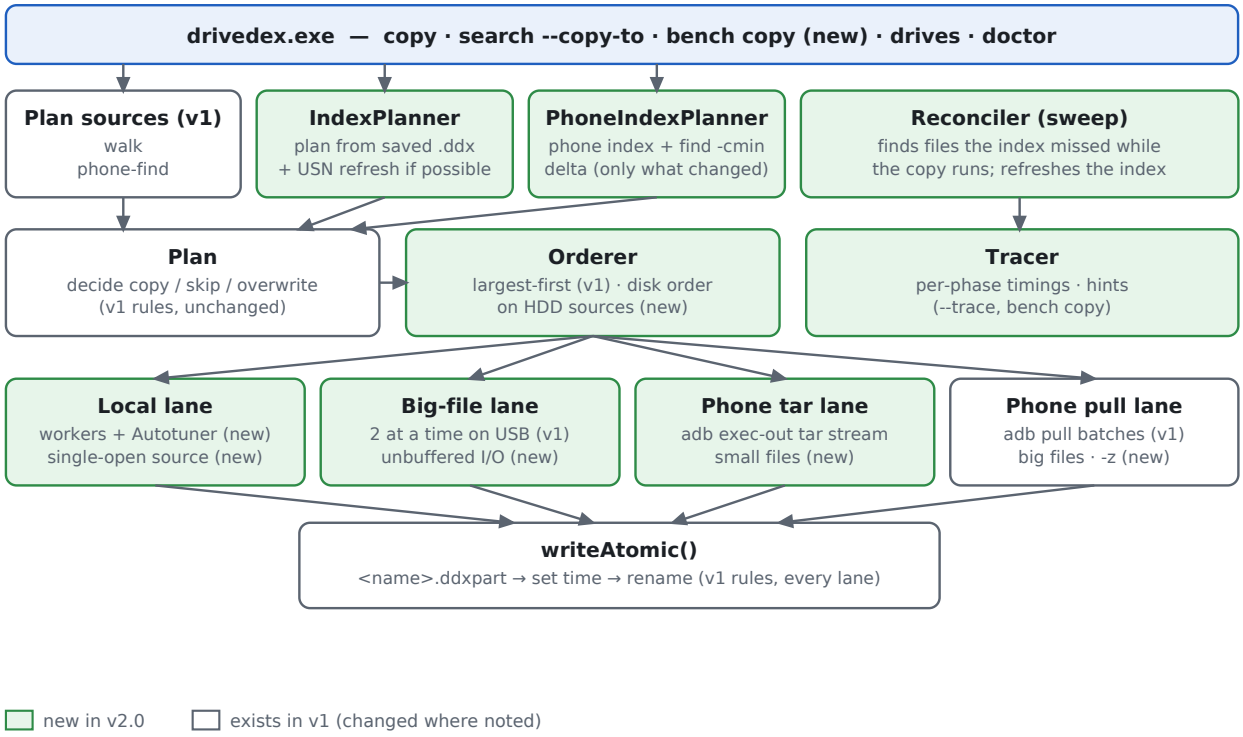
Stage	Name	What the owner gets	Expected effect (hypothesis — to be measured)
13	Copy tracer + bench copy	`copy --trace` shows where time goes; `bench copy` times Explorer, robocopy, v1-mode and v2	No speed change; makes every later claim provable
14	Phone link check	“Phone connected at USB 2.0 (480 Mbps) — max ≈ 40 MB/s”	Diagnosis; avoids chasing an impossible speed
15	Phone: plan from index + delta	Phone copy starts without a full `find`; only changed files are listed	Saves the whole listing time (seconds to ~1 min per 50k files)
16	Phone: tar stream lane	Small phone files arrive in one continuous stream	Largest gain for many small files: ≥1.5× vs v1 is the target
17	Local: plan from index + sweep	Drive copies start at once; a sweep catches new files	Removes listing wait on flash/USB; NTFS + admin: also on HDD
18	Local fast paths	Single-open source, unbuffered big files, media-true verify, overhead hints	Fewer round trips; no cache stalls on big files
19	Autotuner	Worker count adjusts itself during the copy	0–30% depending on the drive
20	Disk-order reads (HDD)	Small files read in the order they sit on the platter	Possibly large on USB hard disks; unknown until measured

Stage	Name	What the owner gets	Expected effect (hypothesis — to be measured)
21	Docs + v2.0.0 release	README, HOW-IT-WORKS, real benchmark table, tag	—

Priority if time runs short: 13 → 14 → 16 → 15 are the phone → PC path the owner asked for. 17–20 speed up drive copies. Each stage is shippable on its own.

4. Architecture of the v2.0 copy engine

v1 has one straight line: *list the source → plan → run workers*. v2.0 turns this into a small pipeline with replaceable parts. The CLI chooses a **plan source**, an **orderer** and the **lanes**; all lanes end in the same **atomic writer**, so the v1 safety guarantees hold for every path.



4.1 Components

Component	Package / file	Responsibility
Tracer	internal/copier/trace (new)	Nil-safe phase timers and counters; produces the --trace report and hints
Plan sources	internal/copier/plan.go (v1), planindex.go (new), phoneplan.go (new)	Produce a Plan: walk, index, phone-find, phone-index(+delta)
Reconciler	internal/copier/reconcile.go (new)	Directory sweep during/after the copy; feeds late jobs; updates the index
Orderer	internal/copier/order.go (new)	largest-first (v1), name, disk (LCN) for HDD sources
Lanes	copy.go (local, big), phonepull.go (v1 code moved), phonetar.go (new)	Move bytes; every lane calls writeAtomic
writeAtomic	internal/copier/write.go (new, extracted from copyFile)	<name>.ddxpart → preallocate → copy → set mtime via handle → close → rename
Autotuner	internal/copier/tune.go (new)	Resizable worker limit, hill-climbing on measured throughput

Component	Package / file	Responsibility
Phone caps	internal/phone/caps.go (new)	Probe tar, find -cmin, adb pull -z support once per serial
Tar stream	internal/phone/tar.go (new)	Start `adb exec-out tar`, return the stream
Phone link	internal/volume/usblink*.go (extended)	USB speed of an adb device, reusing the v1 hub queries
Bench	internal/benchcopy (new) + internal/cli/benchcopy.go	Runs engines in rotation, medians, Markdown row

4.2 How a copy runs in v2.0

1. **Resolve** source and destination (v1 code). Detect source kind: folder on a drive, or phone`:`.
2. **Choose the plan source** (rules in 4.3). Print one line saying which and why, e.g. `Planned from the index of SANDISK 64GB (made 3 min ago) — 41,200 files`.
3. **Plan**: v1 `finalize()` unchanged — conflict policy, destination stats in parallel, FAT32 and free-space checks.
4. **Order** the small-file queue (largest-first, or disk order on an HDD source).
5. **Start lanes** and, if the plan came from an index, the **Reconciler**. Late jobs go through the same `decide()` and join the queue; the progress total grows ("+37 found while copying").
6. **Autotuner** adjusts the local worker limit every window (unless `--workers N`).
7. **Finish**: v1 summary, plus the trace report when `--trace` is on, plus an index refresh from what the sweep or delta saw.

4.3 Choosing the plan source

Source	Condition	Plan source	Reconcile
Phone	phone index exists, `find -cmin` works	phone index + delta (`find -cmin -M`)	none needed (delta covers it)
Phone	no index or no `-cmin`	phone-find (v1)	—
Drive, NTFS	index from mft mode with metadata, process elevated	USN refresh, then index	none needed
Drive, flash/SSD	index younger than `index_plan_max_age` (15 min), ≥95% entries have size+time	index	sweep in parallel with the copy
Drive, HDD (no USN)	any	walk (v1)	— (a parallel sweep would cause seeks)
Anything	`--plan walk` or `--fresh`	walk / phone-find (v1)	—

Correctness before speed. A plan from an index can be stale. The rules above guarantee that every file present at the end of the sweep or delta is either copied or reported. A file that vanished after planning is reported as *vanished* (not a failure); a file whose size/time changed is copied with its real content.

5. Shared contracts (Go types, config, flags)

Names below are binding unless Claude Code finds a clash with existing code — then record the rename in PROGRESS.md.

5.1 Tracer — internal/copier/trace

```
// Package trace measures where a copy spends its time. A nil *Recorder is valid
// and does nothing, so the copy engine can call it unconditionally at zero cost.
package trace

type Phase uint8

const (
    PlanList    Phase = iota // listing the source: walk, find, or reading the index
    PlanDst                  // checking what already exists at the destination
    SrcOpen                  // opening the source (and reading its size/attributes)
    DstCreate                // creating <name>.ddxpart
    Prealloc                 // reserving space
    Data                     // reading + writing the bytes
    SetTime                  // setting the modification time
    Close                    // closing the destination (Windows flushes metadata here)
    Rename                   // .ddxpart -> real name
    Verify                   // --verify re-reads
    PhoneList                // find / delta find on the phone
    PhoneStream              // waiting for bytes from adb (tar or pull)
    Place                    // phone: staging -> final name
    NumPhases
)

type Mark struct{ t int64 } // monotonic nanoseconds

type Recorder struct {
    ns      [NumPhases]atomic.Int64
    count   [NumPhases]atomic.Int64
    perFile [24]atomic.Int64 // histogram of per-file total time, log2 buckets of microseconds
    bytes   atomic.Int64
    files   atomic.Int64
}

func (r *Recorder) Start() Mark                  // nil-safe
func (r *Recorder) End(p Phase, m Mark)          // nil-safe
func (r *Recorder) FileDone(total time.Duration, bytes int64)
func (r *Recorder) Report(elapsed time.Duration, ctx Context) Report

// Context is what the hint rules need to know about the copy.
type Context struct {
    SrcKind, DstKind string  // "usb-flash", "usb-hdd", "ssd", "hdd", "phone", "unknown"
    LinkMBps         float64 // from the USB link check; 0 = unknown
    Workers          int
}
```

5.2 Plan sources and the reconciler — internal/copier

```
// PlanSource produces a Plan. Name() is printed so the owner sees what was used.
type PlanSource interface {
    Name() string // "walk" | "index" | "usn+index" | "phone-find" | "phone-index"
    Plan(ctx context.Context, dstRoot, policy string, workers int) (*Plan, error)
}

type Change uint8

const (
    Added Change = iota // in the source, not in the plan
    Modified            // size or mtime differ from the plan
    Vanished            // in the plan, no longer in the source
)

// Reconciler finds what a plan from an index missed. Emit is called from the
// sweep goroutine; the copier makes it safe to call while workers run.
type Reconciler interface {
    Sweep(ctx context.Context, emit func(j Job, c Change)) error
}

// Plan gains two fields (everything else from v1 is unchanged):
type Plan struct {
    // ... v1 fields ...
    Source string            // PlanSource.Name()
    Late   <-chan LateJob    // nil unless a Reconciler runs
}
```

5.3 The shared atomic writer

```
// writeAtomic streams r into dst through dst+".ddxpart", sets the modification
// time through the open handle, closes, and renames into place. Used by EVERY
// lane (local, big-file, tar, and the pull lane's placing step) so the v1
// guarantee "a real name only ever appears for a complete file" holds everywhere.
func writeAtomic(ctx context.Context, dst string, r io.Reader, size, mtime int64,
    buf []byte, mode ioMode, tr *trace.Recorder) (written int64, err error)

type ioMode uint8 // buffered (v1) | unbuffered (stage 18, big files only)
```

5.4 Phone — internal/phone

```
// Caps is what this phone's shell supports. Probed once per serial per process.
type Caps struct {
    Tar          bool     // toybox/busybox tar can create to stdout
    FindCmin     bool     // find supports -cmin (delta listing)
    PullCompress []string // e.g. ["lz4","zstd","brotli"]: adb client AND device support it
}

func (a *ADB) Probe(ctx context.Context, serial string) (Caps, error)

// TarStream runs "cd <root> && tar -cf - -- <rels...>" on the phone through
// "adb exec-out" and returns the raw tar stream. wait() returns the exit error.
func (a *ADB) TarStream(ctx context.Context, serial, root string, rels []string) (
    stream io.ReadCloser, wait func() error, err error)

// DeltaCommand lists files changed in the last `minutes` minutes (same output
// format as StatCommand, so the v1 parser is reused).
func DeltaCommand(root string, minutes int) string
```

5.5 Autotuner — internal/copier/tune.go

```
// Limit is a resizable semaphore: workers Acquire before each file. Set() may
// shrink or grow it at any time; shrinking never interrupts a file in progress.
type Limit struct{ /* mutex + cond, or channel of tokens */ }
func (l *Limit) Acquire(ctx context.Context) error
func (l *Limit) Release()
func (l *Limit) Set(n int)

type Tuner struct {
    Min, Max, Start int           // bounds; Max is 4 for an HDD at either end, else 16
    Window          time.Duration // 2s
    Hysteresis      float64       // 0.05 = a change must win by 5% to be kept
}
func (t *Tuner) Run(ctx context.Context, l *Limit, sample func() (files, bytes int64), log func(string))
```

5.6 New configuration keys (config.toml)

```
# --- v2.0 copy settings (drivedex config edit) ---
copy_plan_from_index = "auto"   # auto | always | never   (see section 4.3)
index_plan_max_age   = "15m"    # an older index is not trusted for planning on its own
copy_autotune        = true     # off when --workers N is given
copy_order           = "auto"   # auto | size | name | disk   (disk = HDD sources only)
copy_unbuffered      = "auto"   # auto | on | off   (files >= 256 MB)
phone_copy_mode      = "auto"   # auto | tar | pull
phone_tar_streams    = 1        # tar streams at once (bench decides if 2 is better)
phone_pull_compress  = "auto"   # auto | off | lz4 | zstd | brotli   (compressible files only)
```

Every key gets a comment in the generated config file (v1 style) and appears in `config show`. **The final "auto" behaviour of each key is set from the stage benchmarks.**

5.7 New and changed CLI surface

Command / flag	Meaning
`copy --trace`	Print the phase report and hints at the end
`copy --trace-file t.json`	Also write the raw report as JSON (for bench and bug reports)
`copy --plan auto|index|walk, --fresh`	Force the plan source; --fresh = walk/find like v1
`copy --order auto|size|name|disk`	Override the orderer
`copy --unbuffered auto|on|off`	Big-file I/O mode
`copy --phone-mode auto|tar|pull`	Force a phone lane
`copy --v1`	Turn every v2 feature off (used by bench as the "drivedex-v1" engine)
`bench copy <src> <dst>`	New command, stage 13
`drives`, `doctor`	Show the phone USB link (stage 14)

6. Stages

Each stage has: Goal · Why · Files · Spec · Commands · Acceptance (container) · Owner to verify (hardware) · Commit. Container tests use fakes exactly like v1 (the fake adb `Runner`, temp folders, generated fixtures).

Stage 13 — Copy tracer and bench copy

Goal. Know exactly where time goes in a slow copy, and have one command that produces honest, repeatable comparisons. **Why.** The owner reports “slow in real use” but there are no real-hardware numbers; without them no later stage can be judged.

Files

- new `internal/copier/trace/trace.go`, `hints.go`, tests
- new `internal/benchcopy/` (engines, rotation, medians, Markdown), tests
- new `internal/cli/benchcopy.go`; changed `cli/copy.go` (flags), `copier/copy.go`, `copier/phone.go` (Start/End calls)

Spec — tracer

- Instrument every phase in section 5.1. A nil recorder must cost nothing measurable: benchmark `BenchmarkCopySmallFilesNoTrace` before/after; the difference must be under 2%.
- Report: elapsed, files/s, MB/s, per-phase total time and share (%), average per file, p50/p95 per-file time from the histogram, worker count, plan source.
- **Hints** (pure functions, unit-tested, at most 3 printed, most important first):
  - *Link-bound*: Data share > 70% and MB/s ≥ 80% of LinkMBps → “the USB link is the limit (USB 2.0 port?) — software can't go faster; try a USB 3 port.”
  - *Per-file overhead*: (DstCreate+Close+Rename+SetTime) > 40% and average file < 256 KB → “creating files is the slow part. Antivirus scanning each new file is a common cause; you may add the destination folder to Windows Security exclusions yourself.” DriveDex never changes security settings.
  - *Listing-bound*: PlanList > 25% of elapsed → “listing took X s; with a fresh index v2 can skip this (index the drive first, or keep the agent running).”
  - *Phone round trips*: phone copy, PhoneStream share low, many files < 1 MB → “phone copy is waiting between files — tar mode helps.”
  - *Seek-bound*: HDD source, MB/s under 20% of the drive's bench speed → “a hard disk is seeking; fewer workers / disk order helps.”

```
Example (illustrative layout, not real numbers):

Copied 18,412 files (6.1 GB) in 4m 52s  -  63 files/s, 21.4 MB/s  -  plan: walk, workers: 8
Where the time went            total     share   per file
  listing the source           41.2 s     14 %
  creating files (.ddxpart)    88.0 s     30 %    4.8 ms
  data (read + write)         122.5 s     42 %
  close + rename               37.1 s     13 %    2.0 ms
Per-file time: p50 9 ms, p95 61 ms
Hints:
  - Creating files is the slow part (avg file 340 KB). Antivirus scanning each new file is a
    common cause; you can add D:\Backup to Windows Security exclusions yourself.
  - Listing took 41 s; a fresh index lets DriveDex skip this.
```

Spec — bench copy

```
drivedex bench copy <src> <dst-folder> [--rounds 3] [--engines list] [--cold] [--markdown]

engines (default: all that apply):
  explorer      Windows Shell copy engine (Shell.Application CopyHere via go-ole)   [experimental]
  robocopy-1    robocopy <src> <dst> /E /R:0 /W:0 /NFL /NDL /NJH /NJS /NP          (single thread)
  robocopy-mt   same + /MT:16
  drivedex-v1   drivedex copy --v1   (same binary, every v2 feature off)
  drivedex      drivedex copy        (defaults)
phone sources (phone:/DCIM/Camera): adb-pull (one 'adb pull -a' of the folder), drivedex-v1, drivedex
```

- Ask y/N before writing (BUILD.md rule 7). Output goes to `<dst>\.drivedex-bench-<yyyymmdd-hhmmss>\<engine>-r<n>`; each run's folder is deleted after it is measured; at the end the bench root is deleted. Delete only paths under that root (assert the prefix).
- **Rotate the engine order every round** (A B C, B C A, C A B) so cache warm-up does not favour one engine. Report the **median** and the spread.
- **`--cold`**: before each run print “Unplug the source drive, plug it back in, press Enter” — the only honest way to clear Windows' cache for a USB source without admin rights. Without --cold, label results “warm cache”.
- **robocopy exit codes 0–7 mean success**; ≥ 8 is a failure. Treat them that way.
- **explorer engine**: `Folder.CopyHere(item, 4|16|512|1024)` (no progress UI, yes to all, no confirm on mkdir, no error UI). CopyHere returns at once: finish = destination file count and byte total equal the source's and unchanged for 2 s; timeout 2× the slowest other engine. If it can't be measured reliably, skip it with a message and tell the owner to time Explorer by stopwatch.
- Every result line records: engine, files, bytes, median seconds, files/s, MB/s, source & destination description (model, file system, connection and USB speed from v1 code). `--markdown` prints rows ready to paste into docs/BENCHMARKS.md.

Acceptance (container)

- Tracer: unit tests for every hint rule (fires / does not fire), histogram buckets, nil-safety, JSON round trip.
- Overhead benchmark < 2% (above).
- bench copy with fake engines: rotation order, median, cleanup only under the bench root (a test tries a malicious path and must be refused), robocopy exit-code mapping, Markdown output.

Owner to verify

- `drivedex copy <your slow case> --trace` — paste the report. **This is the first real diagnosis of “slow in real use”.**
- `drivedex bench copy E:\SomeFolder D:\BenchTmp --rounds 3 --cold --markdown` on the pen drive, and `bench copy phone:/DCIM/Camera D:\BenchTmp` — paste the rows.

Commit: `stage 13: copy tracer and bench copy`

Stage 14 — Phone USB link check

Goal. Tell the owner the phone's real USB speed before a long copy. **Why.** Many phones — including recent mid-range ones — have USB 2.0 ports (≈35–40 MB/s ceiling). A slow phone copy may be the port, not DriveDex.

Spec

- Map adb serial → USB device: walk present USB devices (`CM_Get_Device_ID`); the instance ID of a phone usually ends with its serial (`USB\VID_18D1&PID_4EE7\<serial>`). For composite devices use the parent (v1 portable code already does this for WPD devices).

- Refactor `volume.USBLinkFor(root)` so its hub-query part takes a device instance: `USBLinkForDevice(instanceID)`; the drive version calls it after finding the disk device.
- Verdicts reuse v1 `Advice()`, plus one phone-specific line: when the phone itself is only USB 2.0-capable → "this phone's port is USB 2.0: about 40 MB/s is the most any software can get."
- Show in: `drivedex drives` (phone row CONNECTION column), the line printed before a phone copy, `doctor`, and the tracer's LinkMBps.
- If the serial can't be matched: "USB speed: can't tell" — never guess.

Acceptance / Owner to verify

- Container: parser and matching tests with recorded instance IDs (Pixel, Samsung, Xiaomi formats); `_other.go` stub.
- Owner: `drivedex drives` with the phone on a USB 3 port and a USB 2 port; paste both.

Commit: `stage 14: phone USB link check`

Stage 15 — Phone: plan from the index, list only what changed

Goal. A phone copy no longer starts with a full `find` of the folder. **Why.** v1 `PlanPhone` always runs the stage-5 scanner even though the agent indexed the phone when it was plugged in (G1).

Spec

- Probe `find -cmin`: `find /sdcard -maxdepth 0 -cmin -1` — exit code 0 and no "unknown option" on stderr means supported. Cache in `Caps`.
- If an index for this serial exists and FindCmin: *M* = minutes since index UpdatedAt + 2 (clock-skew margin). Run `DeltaCommand(remote, M) = find <remote> -type f -cmin -M -exec stat -c '%s|%Y|%n' {} + 2>/dev/null`. Use **-cmin**, not -mmin: a moved/renamed file keeps its mtime but its ctime changes, so -mmin would miss it.
- Plan = index entries under `remote` ∪ delta entries (delta wins on conflict). Files deleted since the index will be reported as *vanished* at transfer time.
- After the copy, upsert the delta into the phone index and save it (atomic, v1 store) — a free refresh.
- Fallbacks to v1 phone-find: no index, no -cmin, index older than 7 days, `--fresh`.
- Print the choice: "Planned from the phone index (made 12 min ago) + 214 changed files."

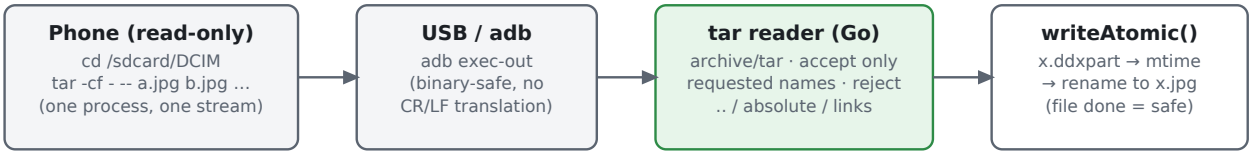
Acceptance / Owner to verify

- Container (fake runner): delta merge, -cmin probe both ways, vanished handling, index upsert saved, fallbacks.
- Owner: take 3 photos after indexing, then `copy phone:/DCIM/Camera D:\x --trace`: the 3 new photos are copied, PhoneList time is a fraction of v1's (compare with `--fresh`).

Commit: `stage 15: phone copy planned from index + delta`

Stage 16 — Phone: tar stream lane (the main phone → PC speed-up)

Goal. Small phone files arrive in one continuous stream instead of one request per file. **Why.** G2: in `adb pull` each file is a separate request/transfer/acknowledge exchange. With thousands of small files, the link spends much of its time waiting.



v1: up to 100 files per 'adb pull', but adb still transfers them one by one (request → file → reply).
v2: up to 2,000 files per tar stream — the phone never waits for the PC between files.

Spec — phone side (read-only)

- Probe tar once per serial with a real round trip: run `adb -s S exec-out "tar -cf - -C /system/etc hosts"` and parse the output with Go's archive/tar. One regular entry named `hosts` → Caps.Tar = true; anything else → false. Most Android 10+ phones ship toybox tar, but never assume it.
- Command per chunk: `adb -s S exec-out "cd <root> && tar -cf - -- 'rel1' 'rel2' … 2>/dev/null"`. Quote with v1 `shellQuote`. Use **exec-out**, never `adb shell`, for binary data (older adb/pty mode turns LF into CRLF and corrupts files).
- Chunks: ≤ 2,000 files and ≤ 24,000 characters of arguments (v1's adb command-line budget). Paths are relative to the folder being copied, so they stay short.
- No compression in the tar lane (photos/videos are already compressed; phone CPUs are slow).

Spec — PC side

- Read with Go `archive/tar` (it handles ustar, GNU long names and PAX).
- **Security**: accept an entry only if (a) its type is a regular file, (b) its cleaned name is in the requested set for this chunk. Reject absolute names, `..`, links, devices. Unknown entries are drained and counted as errors. (A compromised or buggy phone must not be able to write outside the destination.)
- Write each entry with `writeAtomic` to `DstRoot\SafeRel(rel)` with the tar header's ModTime (seconds, same as the index). File done = renamed = safe for resume.
- Requested files that never arrived (deleted, unreadable) → retry once via the pull lane, then report *vanished* or failed with the reason.
- A stream error mid-file: discard that `.ddxpart`, re-plan the rest of the chunk (already-renamed files are skipped by the v1 rules), retry once with tar, then fall back to pull for that chunk.

Spec — lanes and scheduling

Item	Lane	Why
Files < 32 MB (BigFileSize)	tar lane, `phone_tar_streams` (default 1) streams	Per-file round trips are what tar removes
Files ≥ 32 MB	pull lane (v1 code), 1 worker while tar runs	Same throughput as tar; resumable per file
Names Windows can't store	pull lane, one at a time (v1 SafeName path)	v1 already handles these correctly
Compressible big files (.txt .log .json .db .csv .xml .html)	pull with `-z <algo>` if Caps.PullCompress	Only when stage-16 bench shows a gain

- Compression probe: the adb client supports `pull -z` if `adb help` mentions it; the device supports an algorithm if `adb -s S features` lists `sendrecv_v2_lz4` / `_zstd` / `_brotli`. Prefer lz4 (cheapest CPU). Verify this on the owner's adb version and record it.
- `phone_copy_mode = auto`: tar if Caps.Tar, else v1 pull. `--phone-mode pull` = exact v1 behaviour.

Acceptance (container)

- Fake runner streams a tar built in the test with Go's tar writer: normal files, long names (> 100 chars), names needing SafeName, an entry with `../evil`, an absolute entry, a symlink, an unrequested file → only requested regular files land, the rest are refused and counted.
- Stream cut mid-file (fake returns io.ErrUnexpectedEOF): no partial file under a real name, re-run completes, byte-identical, no `.ddxpart` left.
- Missing files → pull fallback → vanished. Tar not available → whole copy uses pull (v1).
- Stress: 5,000 files of 0 B – 2 MB through the fake, cancelled at random points 4×, then completed: identical.

Owner to verify — the decisive test

- Folder with ≥ 5,000 files under 1 MB (e.g. `phone:/Android/media/com.whatsapp/WhatsApp/Media` or Pictures/Screenshots): `bench copy phone:<folder> D:\BenchTmp --rounds 3` (engines adb-pull, drivedex-v1, drivedex). **Target: drivedex ≥ 1.5× faster than drivedex-v1.**
- Camera folder with large videos: drivedex within ±5% of drivedex-v1 (no regression).
- Unplug the phone mid-copy, replug, re-run: resumes, nothing corrupted.
- Try `phone_tar_streams = 2` once; keep 2 only if it wins by ≥ 10%.

Commit: `stage 16: phone tar stream lane`

Stage 17 — Local copy: plan from the index + reconciling sweep

Goal. `drivedex copy E:\Photos D:\Backup` starts copying immediately when E: has a fresh index. **Why.** G1: v1 walks the whole source first (“Reading the file list…”), on the slowest device in the chain.

Spec

- Find the index with v1 `index.LocateScope(indexes, env, srcAbs)`. Build a parent → children map once (O(n)) and collect every entry under the scope folder into Jobs (size and mtime from the index, Rel from `RelPathCached`).
- Use the index only if the rules in section 4.3 allow it. NTFS turbo indexes made without `--with-metadata` have `FlagNoMeta` entries: if more than 5% of the scope lacks size/time, fall back to walk and print why.
- **NTFS + elevated + journal**: run the v1 USN refresh first (milliseconds), then plan; no sweep.
- **Flash/SSD source**: start the copy at once and run the **Reconciler** in parallel: a directory walk with 2 workers using `os.ReadDir` (Windows returns size and time from the directory listing — no per-file open). Compare with the plan by Rel: Added → late job; Modified → if not yet copied, update the item; if already copied, copy again; Vanished → nothing (the copy reports it).
- **HDD source without USN**: walk (v1). A parallel sweep would make the head seek between listing and reading.
- Late jobs pass through v1 `decide()` and the FAT32 check; free-space is re-checked for the added bytes. The progress line shows “+N found while copying”.
- At the end, the sweep result refreshes the drive's index (atomic save) — the next copy plans from a fresh index.
- `search --copy-to` already copies from index results; make it use the same writer and lanes.

Acceptance (container)

- Fixture indexed, then modified (add 50 files, change 20, delete 10, rename 1 folder) → copy with `--plan index` → destination equals the *current* source (diff -r), the 10 deletions reported as vanished, not failed; index refreshed.

- FlagNoMeta-heavy index → falls back to walk with the message. Old index (> max age) → walk.
- Ctrl+C during the sweep, re-run → completes, identical. `-race` clean.

Owner to verify

- Pen drive with ≥ 20,000 files, indexed by the agent: time-to-first-file (from the trace) under 1 s vs v1's listing time; total time ≤ v1 (bench copy, --cold).
- Add files to the pen drive after indexing; copy; they are copied ("+N found while copying").

Commit: `stage 17: local copy planned from index with sweep`

Stage 18 — Local fast paths

Goal. Cut per-file round trips and remove big-file cache stalls. Each item is measured separately with the tracer; any item that does not help is left off by default.

18a. Single-open source (G4)

- Windows: open the source once with `windows.CreateFile(GENERIC_READ, share read|write|delete, OPEN_EXISTING, FILE_FLAG_OPEN_REPARSE_POINT | FILE_FLAG_SEQUENTIAL_SCAN)`, then `GetFileInformationByHandle`: if the reparse-point attribute is set → skip as not regular (same rule as v1's Lstat check — never follow links); size and mtime come from the handle.
- This replaces `os.Lstat` + `os.Open` in `copyFile`. Go's Lstat on Windows is a separate path lookup (GetFileAttributesEx), so this saves one metadata round trip per file — the tracer's SrcOpen phase shows how much on the owner's drive.
- `_other.go` keeps the v1 code path.

18b. Unbuffered I/O for big files (G5)

- Files ≥ 256 MB with `copy_unbuffered = auto`: open source and destination with `FILE_FLAG_NO_BUFFERING` (what `robocopy /J` does).
- Rules for unbuffered I/O: buffer address and every read/write size must be multiples of the volume sector size (use 4096-byte alignment; allocate buffer+4096 and slice at an aligned offset; use 8 MB buffers). Write the last partial block padded to a full sector, then set the true size with `SetFileInformationByHandle(FileEndOfFileInfo)`.
- Keep preallocation, .ddxpart, mtime-by-handle and rename exactly as v1.
- Default for "auto" decided by the bench: one 4 GB file and a 20 GB set to a USB 3 drive; keep it on only if it is ≥ 10% faster or removes the "fast then stalls" pattern (visible in the progress speed).

18c. Verify that checks the media (G6)

- `--verify` re-reads the *destination* with `FILE_FLAG_NO_BUFFERING` so the checksum comes from the disk, not RAM. Update the flag's help text accordingly.
- Only the destination read needs this; the source checksum can be computed while copying (tee the CRC during the copy) — saving one full source re-read.

18d. Overhead hints

- The tracer's per-file-overhead hint (stage 13) becomes actionable: when it fires, `doctor` explains how to add an exclusion in Windows Security *manually*. DriveDex never changes it.

18e. Experiment, off by default: direct writes for brand-new folders

- When the destination folder was created by this run, write small files (< 1 MB) directly to their final names and record completed files in `<dst>\.drivedex-resume` (append-only, fsync every 1,000 entries). On resume, files not in the journal are re-copied. Saves one rename per file.

- Ship only if the 5,000-small-file bench is ≥ 15% faster; otherwise keep the code behind `--experimental-direct` or drop it.

Acceptance

- Container: aligned-buffer tests (sizes 0, 1, 4095, 4096, 4097, 256 MB+1 using sparse files), reparse-point skip (junction + symlink on Windows CI), verify with tee'd CRC, resume journal crash test (kill mid-write).
- Owner: bench rows for 18a, 18b, 18e with the decision written in PROGRESS.md.

Commit: `stage 18: local fast paths`

Stage 19 — Autotuner

Goal. The worker count finds its own best value during a copy. **Why.** G3: 2 / 4 / 8 fixed workers is wrong for many real drives — cheap pen drives slow down with parallel writes, fast SSDs want more.

Spec

- Workers acquire a slot from `Limit` before each file (section 5.5). The big-file lanes are not tuned (they stay at 2 on USB).
- Every 2 s window, measure score = files/s while the small-file queue dominates, otherwise MB/s. Use the median of the last 3 windows to smooth noise.
- Hill-climb: try Start+2; if the score improves by > 5%, keep and try +2 again; if worse, go back and try −1; settle when neither direction wins; re-probe every 30 s (drives change behaviour when their cache fills).
- Bounds: Min 1; Max 4 if either end has a seek penalty, else 16.
- Off when `--workers N`, `copy_autotune = false` or `--v1`. Each change is logged in the trace ("workers 8 → 10: +7% files/s").

Acceptance

- Container: a simulated device (score function with a known optimum, plus noise) → the tuner settles within ±1 of the optimum in < 10 windows; shrinking never interrupts a running file; -race clean.
- Owner: bench with autotune on vs `--workers auto` (v1) on a pen drive and the SSD; keep on only if not slower anywhere.

Commit: `stage 19: worker autotuner`

Stage 20 — Disk-order reads for hard-disk sources

Goal. On a spinning disk, read small files in the order they sit on the platter. **Why.** G7: name or size order makes the head jump; disk order turns many seeks into short forward moves.

Spec

- Only when the source has a seek penalty (v1 `volume.SeekPenalty`) and is NTFS; `copy_order = auto` or `--order disk`.
- For each planned small file, get its first cluster: `FSCTL_GET_RETRIEVAL_POINTERS` on a handle opened with `FILE_READ_ATTRIBUTES` only (no admin needed). First extent's LCN is the sort key. Files with no extents (tiny files stored inside the MFT record) get key 0 — they live in the MFT and are read first, together.
- Do the LCN pass with 2 workers *before* copying and time it (trace phase PlanList). If it costs more than 10% of the estimated copy time, abort it and use size order.

- Small-file queue is dispatched in LCN order with the tuned worker count (≤ 2 on HDD). Big files keep their lane.
- Fallback (no NTFS / FSCTL fails): v1 order. Optional cheaper key to test: the NTFS file reference number (FRN), which the MFT-mode index already stores as the entry ID — files created together are usually close on disk. Bench both if time allows.

Acceptance

- Container: ordering logic with fake LCNs; MFT-resident files first; abort rule; non-NTFS fallback.
- Owner: USB hard disk, folder with ≥ 10,000 small files, bench --cold: disk order vs size order. Keep “auto” on only if it wins.

Commit: `stage 20: disk-order reads on hard disks`

Stage 21 — Docs, benchmarks and the v2.0.0 release

- README: new “What's new in 2.0” section, updated copy section, honest numbers only from BENCHMARKS.md.
- HOW-IT-WORKS.md: beginner sections on tar streams vs adb pull, planning from an index, the autotuner, unbuffered I/O, disk order — same style as v1.
- BENCHMARKS.md section 2 filled with the owner's rows (drive, port, file system, cold/warm).
- CHANGELOG.md (new) listing every v2 feature, its switch and its default.
- PROGRESS.md: v2 section complete, deviations, decisions with their bench evidence.
- Ask the owner, then tag `v2.0.0` and publish the release zips (v1 build.ps1).

Commit: `stage 21: docs and v2.0.0`

7. Benchmark protocol (how “faster” is decided)

1. Same PC, same ports, laptop on charger, other heavy apps closed. Record: PC model, Windows version, drive make/model/capacity, file system, port (USB 2.0 / 3.x, from `drivedex ports`), cable.
2. Use `bench copy --rounds 3 --cold` for drive sources; phone sources: 3 rounds, phone unlocked, screen on, not charging-only mode.
3. Report the median; a win must exceed the spread between rounds, otherwise it's “no difference”.
4. Always include `drivedex-v1` (same binary with `--v1`), so a gain can't come from something other than the v2 feature.
5. Test sets (create once, keep): (A) 5,000+ files under 1 MB; (B) 300–500 camera photos 2–8 MB; (C) a few files of 1–4 GB; (D) 10,000+ mixed files on a USB hard disk.

Template for docs/BENCHMARKS.md (times are medians in seconds):

Date	Test set	Source (model, FS, port)	Destination	Files / size	Expl.	robo MT	v1	v2	Notes

8. Risks, fallbacks and what is out of scope

8.1 Risks

Risk	Mitigation
Phone has no tar or an unusual tar	Probe; fall back to pull per copy or per chunk; owner can force --phone-mode pull
Malicious/buggy tar stream writes outside the destination	Accept only requested regular-file names; reject .., absolute, links (tested)
Index is stale → files missed	Rules in 4.3: sweep, USN, delta, or walk; vanished/added reported
Unbuffered I/O alignment bugs corrupt data	Alignment tests incl. odd sizes; --verify reads media; off unless bench wins
Autotuner oscillates	Median of 3 windows, 5% hysteresis, bounds, re-probe only every 30 s
-cmin unsupported or clock skew	Probe; +2 min margin; fallback to full find
Disk-order pass too slow	10% budget, then abort to size order
Benchmarks flattered by cache	--cold replug prompts, engine rotation, medians, v1 baseline in the same binary

8.2 Out of scope for v2.0 (candidates for v2.1)

- Copying *out of* MTP devices (iPhone, cameras, phones without USB debugging). v1 indexes them but never copies; doing it through the Windows Shell is slow and needs a device for testing.
- Reading LCNs from the raw MFT (admin, complex parsing) — only if stage 20 shows disk order is worth it.
- Wireless ADB (adb over Wi-Fi) — usually slower than USB; possible convenience feature later.
- PC → phone copies (would break the read-only-on-phone rule; needs a separate design).
- Compressing the tar stream on the phone (gzip/zstd) — only if a bench on compressible folders justifies it.

Appendix A — PROGRESS.md section to add

```
## v2.0 (BUILD-V2.pdf)
Legend: [x] done and verified - [~] code done, needs owner hardware - [ ] not started

- [ ] 13 Copy tracer + bench copy
- [ ] 14 Phone USB link check
- [ ] 15 Phone: plan from index + delta
- [ ] 16 Phone: tar stream lane
- [ ] 17 Local: plan from index + sweep
- [ ] 18 Local fast paths (18a single-open, 18b unbuffered, 18c verify, 18d hints, 18e experiment)
- [ ] 19 Autotuner
- [ ] 20 Disk-order reads (HDD)
- [ ] 21 Docs + v2.0.0

### Defaults decided by benchmark
| Setting | Default | Evidence (BENCHMARKS.md row) |
|---|---|---|
| phone_copy_mode | | |
| phone_tar_streams | | |
| phone_pull_compress | | |
| copy_plan_from_index | | |
| copy_unbuffered | | |
| copy_autotune | | |
| copy_order | | |

### Owner to verify (v2)
(one bullet per stage, copied from BUILD-V2.pdf)
```

Appendix B — Files touched (map for Claude Code)

File	Change
internal/copier/copy.go	Extract writeAtomic to write.go; Limit-based workers; trace calls; late-job queue
internal/copier/plan.go	PlanSource interface; Plan.Source, Plan.Late; walk source wraps PlanLocal
internal/copier/phone.go	Split: phoneplan.go (index + delta), phonepull.go (v1 pull lane), phonetar.go (new)
internal/copier/{planindex,reconcile,order,tune, write}.go	New (stages 17, 17, 20, 19, 18)
internal/copier/fileops_windows.go	Single-open source, unbuffered I/O, LCN query (+ _other stubs)
internal/copier/trace/	New package (stage 13)
internal/phone/{caps,tar}.go, scanner.go	Probe, TarStream, DeltaCommand
internal/volume/usblink*.go	USBLinkForDevice; phone serial matching (stage 14)
internal/benchcopy/, internal/cli/benchcopy.go	New (stage 13)
internal/cli/copy.go	New flags; plan-source choice + message; --v1
internal/cli/drives.go, doctor.go	Phone link speed
internal/config/config.go	New keys with comments (section 5.6)
README.md, docs/*.md, CHANGELOG.md, PROGRESS.md	Stage 21 (PROGRESS every stage)

Appendix C — Glossary for the owner

Term	Plain meaning
ADB / adb pull	Android Debug Bridge: a USB channel to the phone. adb pull copies files, but asks for and confirms each one separately.
tar stream	Many files packed back-to-back into one continuous stream of bytes; unpacked as it arrives. No waiting between files.
exec-out	adb mode that passes a program's output through untouched — safe for binary data.
MTP	The protocol Explorer uses for phones; very slow per file and per folder.
Index plan	Using the saved list of files (the .ddx) as the copy plan instead of listing the drive again.
Reconciler / sweep	A quick re-listing during the copy that catches files added or changed since the index was made.
USN journal	NTFS's diary of changes; lets DriveDex update an index in milliseconds (needs admin).
LCN	Logical cluster number: where a file's data starts on the disk. Sorting by it = reading in platter order.
Unbuffered I/O	Reading/writing straight to the disk, skipping Windows' RAM cache — steadier for huge files.
Hill-climbing autotuner	Try a few more workers; keep the change if it's faster, undo it if it's slower; repeat.
Cold cache	First read after a replug, so Windows can't answer from RAM; the only fair benchmark.